

Homework 4 (NOT Graded)

Professor: Dr. Mudassir Shabbir

Deadline: N/A.

This homework contains a set of practice problems and **no submission is required**.

1. *The greedy approach to coin change breaks for non-standard denominations, which arises in currency design and dynamic programming motivation.*

A country uses coin denominations $\{1, 4, 6\}$.

- (a) Show that the greedy algorithm (always pick the largest coin that fits) does *not* produce the optimal solution when making change for 8.
- (b) Give an example denomination set $\{1, c, d\}$ where greedy is *optimal* for all amounts, and explain why.
- (c) Prove that for denominations $\{1, 2, 5, 10, 25\}$ (US cents), the greedy algorithm is always optimal.¹

2. *Interval scheduling appears in resource allocation, CPU scheduling, and calendar management.*

You are given n jobs. Job i has a **deadline** d_i (it must finish by time d_i) and a **penalty** w_i if it misses its deadline. Each job takes exactly one unit of time. You can only do one job at a time and you want to **minimize the total penalty**.

- (a) Prove that there always exists an optimal schedule with no idle time before all jobs are done, i.e., an optimal schedule is a permutation of all n jobs.
- (b) Consider the greedy strategy: schedule jobs in *decreasing* order of penalty, placing each job as late as possible before its deadline (or mark it as missed if no slot is available). Give a small example (4 jobs) and trace this algorithm.
- (c) Prove or disprove: sorting by w_i/d_i (penalty-to-deadline ratio) in decreasing order always gives an optimal solution.

3. *Fractional knapsack models resource allocation when items can be partially taken, such as loading cargo onto a ship.*

You have a knapsack of capacity W . There are n items; item i has weight $w_i > 0$ and value $v_i > 0$. Unlike the 0/1 knapsack, you may take a *fraction* of any item.

- (a) Describe a greedy algorithm for the fractional knapsack problem.
- (b) Prove your algorithm is optimal using an exchange argument.²
- (c) Show by example that the same greedy strategy fails for the 0/1 knapsack (where each item is taken entirely or not at all).

4. *Greedy algorithms can appear correct but fail subtly. This is one of the most instructive failure modes in algorithm design.*

Consider a **weighted** activity selection problem. You have n intervals, each with a start time s_i , finish time f_i , and weight w_i . You want to select a set of non-overlapping intervals with **maximum total weight**.

- (a) Show that the greedy strategy “always pick the non-overlapping interval with the smallest finish time” fails for weighted intervals.
- (b) Show that “always pick the non-overlapping interval with the highest weight” also fails.
- (c) Show that “always pick the non-overlapping interval with the best weight-to-duration ratio” also fails. Provide a concrete counterexample for each.

¹Hint: Show that for any amount, the greedy solution uses no more coins than any other solution, by considering each denomination class separately.

²Suppose an optimal solution differs from the greedy solution. Find the first item where they disagree and show swapping towards the greedy choice cannot decrease the value.

5. *Union-Find models dynamic connectivity, which arises in network routing, image segmentation, and Kruskal's MST algorithm.*

Suppose you have $n = 8$ elements $\{1, 2, 3, 4, 5, 6, 7, 8\}$, each initially its own component. Perform the following sequence of operations using **union by rank** and **path compression**. After each Union, draw the resulting forest (showing parent pointers and ranks).

- (a) **Union**(1, 2), **Union**(3, 4), **Union**(5, 6), **Union**(7, 8)
- (b) **Union**(1, 3), **Union**(5, 7)
- (c) **Union**(1, 5)
- (d) **Find**(8) — trace the path taken and show the state of the data structure after path compression.
- (e) How many distinct components remain after all operations? Which elements are in each component?

6. *This problem explores why union by rank is crucial for efficiency.*

Consider running $n - 1$ **Union** operations on n elements using **naive union** (attach the second tree's root under the first tree's root, regardless of rank), and then n **Find** operations.

- (a) Give a sequence of $n - 1$ **Union** operations that causes the resulting tree to be a chain (path) of length n .
- (b) For the chain you constructed, what is the cost of n **Find** operations on the deepest element?
- (c) Now prove that with **union by rank**, no tree can have height greater than $\lfloor \log_2 n \rfloor$.³
- (d) Conclude that with union by rank (and no path compression), any sequence of m operations on n elements costs $O(m \log n)$ total.

7. *Union by size is an alternative to union by rank that uses subtree node counts instead of height estimates.*

Define **union by size**: always attach the smaller tree under the root of the larger tree; break ties arbitrarily. Track $\text{size}[i]$ = number of nodes in the subtree rooted at i .

- (a) Write pseudocode for **Union** and **Find** using union by size (no path compression).
- (b) Prove that with union by size, the depth of any node is at most $\lfloor \log_2 n \rfloor$.⁴
- (c) Compare union by size and union by rank: do they always produce identical trees? If not, give an example where they differ.

8. *The running median is used in streaming data analysis, real-time signal processing, and financial tick data.*

You receive a stream of integers one at a time and after each insertion you must report the **median** of all integers seen so far.

- (a) Describe a data structure using **two heaps** (a max-heap and a min-heap) that supports **Insert** and **GetMedian** each in $O(\log n)$ time, where n is the number of elements inserted so far.
- (b) Prove the correctness of your data structure: what invariant do you maintain between the two heaps?
- (c) Trace your data structure on the input stream: 5, 3, 8, 1, 9, 2, 7. After each insertion, show the state of both heaps and the current median.

9. *Merging sorted sequences efficiently is fundamental in external sorting and distributed computing.*

You have k sorted arrays, each of length m , for a total of $n = km$ elements. You want to merge them into a single sorted array.

- (a) Show that the naive approach of merging arrays pairwise (merge array 1 with 2, then merge result with 3, etc.) takes $O(nk)$ time in the worst case.
- (b) Describe an algorithm using a **min-heap** of size k that merges all arrays in $O(n \log k)$ time.

³Use induction: show that a tree of rank k must contain at least 2^k nodes.

⁴Show that when a node's subtree doubles in size, its depth increases by at most 1.

10. The k -th smallest element problem arises in order statistics, database query optimization, and approximate algorithms. We designed a linear-time algorithm using the median-of-medians earlier, but a simpler $O(n \log k)$ algorithm using a heap is at times more practical.

- (a) Show that BuildHeap gives an $O(n)$ algorithm to find the minimum element. Can you find the second-minimum in $O(\log n)$ additional time? The k -th minimum in $O(k \log n)$ time?
- (b) Suppose the array is already a *max-heap*. What is the minimum number of elements you must examine to find the k -th largest element?

11. Priority queues with a decrease-key operation are essential for Dijkstra's shortest path and Prim's MST algorithms.

A **decrease-key** operation takes a node v in the heap and a new key $k' < \text{key}[v]$, updates the key, and restores the heap property.

- (a) Describe how to implement **DecreaseKey** on a binary min-heap in $O(\log n)$ time. (Assume you have a direct pointer to v 's position in the heap.)
- (b) In Dijkstra's algorithm on a graph with V vertices and E edges, how many **Insert** and **DecreaseKey** operations are performed in total? What is the total running time using a binary heap?
- (c) Without a pointer into the heap, one common workaround is to use *lazy deletion*: instead of updating an existing entry, insert a new entry with the smaller key and mark the old one as invalid. Describe this approach and analyze its time and space complexity for Dijkstra's algorithm.

12. This problem connects the two main data structures from this unit.

Kruskal's algorithm finds a Minimum Spanning Tree (MST) by sorting edges by weight and greedily adding each edge that does not create a cycle.

- (a) Describe how Union-Find is used in Kruskal's algorithm to detect cycles efficiently.
- (b) Suppose instead of Union-Find, you use a simple BFS/DFS to check each time whether an edge creates a cycle. What is the total running time of this modified Kruskal's?
- (c) A graph has $V = 1000$ vertices and $E = 5000$ edges. Estimate the number of Union-Find operations performed during Kruskal's, and compare the total time using Union-Find vs. BFS cycle detection.
- (d) Now suppose you want to build a *maximum* spanning tree instead of a minimum one. How would you modify Kruskal's algorithm? Does Union-Find still apply without modification?

13. This sequence of questions builds a complete proof of Kruskal's algorithm's correctness from first principles. Throughout, assume all edge weights are distinct.

We work with a connected undirected weighted graph $G = (V, E, w)$ where $w : E \rightarrow \mathbb{R}$ assigns a distinct real weight to each edge.

Definitions.

- A **spanning tree** of G is a subgraph $T \subseteq E$ such that (V, T) is connected and acyclic. A **minimum spanning tree (MST)** is a spanning tree of minimum total weight $\sum_{e \in T} w(e)$.
 - A **cut** of G is a partition of V into two non-empty sets $(S, V \setminus S)$. An edge $e = (u, v)$ **crosses** the cut if exactly one of u, v lies in S .
 - A **forest** is an acyclic graph (not necessarily spanning). Kruskal's maintains a growing forest F that starts empty and ends as a spanning tree.
 - Given a spanning tree T and an edge $e = (u, v) \in T$, the **fundamental cut** of e with respect to T is the cut $(S_e, V \setminus S_e)$ where S_e is the set of vertices reachable from u in $T - e$. Every non-tree edge $f \notin T$ that crosses this cut can be swapped with e to produce another spanning tree.
- (a) **(Removing an edge from a cycle.)** Suppose edge e lies on a cycle C in graph G . Prove that $G - e$ is still connected between the endpoints of e , and hence $G - e$ has the same number of connected components as G . Conclude: if T is a spanning tree and $f \notin T$, then $T \cup \{f\}$ contains *exactly one* cycle.

- (b) **(Removing an edge from a tree.)** Let T be a spanning tree of G and let $e = (u, v) \in T$. Prove that $T - e$ has exactly two connected components S_e (containing u) and $V \setminus S_e$ (containing v), which are precisely the two sides of the fundamental cut of e .
- (c) **(Swapping edges preserves spanning trees.)** Let T be a spanning tree and let $e \in T$, $f \notin T$. Suppose f crosses the fundamental cut of e . Prove that $T' = T - \{e\} + \{f\}$ is also a spanning tree of G .
(Hint: show T' is connected and has $|V| - 1$ edges, hence acyclic.)
- (d) **(Cycle property.)** Let C be any cycle in G and let e^* be the unique maximum-weight edge of C . Prove that e^* is **not** in any MST.
(Hint: suppose some MST T contains e^* . By part (b), removing e^* defines a fundamental cut $(S, V \setminus S)$. Since C is a cycle and e^* crosses the cut, some other edge $f \in C$ also crosses it. Use part (c) and the weight of f to derive a contradiction.)
- (e) **(Cut property.)** Let $(S, V \setminus S)$ be any cut of G and let e^* be the unique minimum-weight edge crossing it. Prove that e^* is in **every** MST.
(Hint: suppose some MST T does not contain e^* . Since T is a spanning tree it must have some other edge f crossing the cut. Apply part (c) and compare $w(e^*)$ with $w(f)$.)
- (f) **(Correctness of Kruskal's algorithm.)** Recall Kruskal's algorithm: process edges in increasing weight order; add edge $e = (u, v)$ to forest F if and only if u and v are in different components of F .
- When Kruskal's considers edge $e = (u, v)$ and decides to add it, let S be the component of F currently containing u . Prove that e is the minimum-weight edge crossing the cut $(S, V \setminus S)$.⁵
 - Use the cut property (part e) to conclude that every edge Kruskal's adds belongs to every MST.
 - Kruskal's terminates with $|V| - 1$ edges forming a spanning tree T^* . Argue that T^* is an MST.
- (g) **(What if weights are not distinct?)** Without distinct weights, an MST may not be unique.
- Give a graph on 3 vertices $\{a, b, c\}$ with equal-weight edges where there are multiple MSTs.
 - Does Kruskal's still always produce *an* MST (not necessarily the unique one)? Argue briefly why or why not.
 - Does the cycle property still hold verbatim ("the maximum-weight edge in any cycle is in *no* MST")? If not, state the correct version.

14. The MST problem has many natural variants. Each one forces you to ask: does the cut property still hold? Does sorting by weight still work? What changes and what stays the same?

- (a) **(Maximum Spanning Tree.)** A **maximum spanning tree** (MaxST) is a spanning tree of maximum total weight.
- State an analog of the **cut property** for maximum spanning trees: which crossing edge of a cut is guaranteed to belong to every MaxST?
 - State an analog of the **cycle property**: which edge in a cycle is guaranteed to be absent from every MaxST?
 - Describe two different ways to adapt Kruskal's algorithm to find a MaxST. Do both work?
 - Prove that negating all edge weights reduces the MaxST problem to the MST problem, i.e., if T^* is an MST of $(G, -w)$, then T^* is a MaxST of (G, w) .
- (b) **(Negative edge weights.)** Suppose all edge weights are strictly negative, i.e., $w(e) < 0$ for all $e \in E$.
- Does Kruskal's algorithm (sort edges in increasing order, greedily add if no cycle) still correctly find the MST? Justify your answer. (Hint: does the proof of correctness rely on weights being positive?)
 - What does the MST intuitively look like when all weights are negative? Which edges does Kruskal's pick first?
 - Now suppose some weights are positive and some are negative. Is there any spanning tree that is guaranteed to have negative total weight? Or could the MST have positive total weight? Give examples for both cases.
- (c) **(Bottleneck Spanning Tree.)** A **bottleneck spanning tree** (BST) is a spanning tree that minimizes the weight of its *heaviest* edge: $\min_T \max_{e \in T} w(e)$.

⁵Every lighter edge was processed earlier. If it was added to F , its endpoints merged into the same component. If it was rejected, both its endpoints were already in the same component. In either case, no lighter edge crosses $(S, V \setminus S)$.

- i. Give an example of a graph where the MST and the BST have different total weights, but the same bottleneck value (heaviest edge weight).
 - ii. Prove that **every MST is also a BST**.⁶
 - iii. Is every BST also an MST? Prove it or give a counterexample.
- (d) (**Minimax path.**) Given a weighted graph G and two vertices s and t , a **minimax path** from s to t is a path P that minimizes the maximum edge weight along the path:

$$\min_{P:s \rightsquigarrow t} \max_{e \in P} w(e).$$

- i. Give a small example (5 vertices) where the minimax path from s to t is *not* the shortest path from s to t .
 - ii. Let T^* be any MST of G . Prove that the unique path from s to t in T^* is a minimax path.⁷
 - iii. Conclude: to answer minimax path queries between any pair of vertices, it suffices to build the MST once in $O(E \log E)$ time, then answer each query by finding the maximum-weight edge on the s - t path in T^* .
- (e) (**Forced and forbidden edges.**) Suppose some edges are *forced* (must appear in the spanning tree) and others are *forbidden* (must not appear).
- i. You are told that edge $e_0 = (u, v)$ **must** be in the MST. Describe how to modify Kruskal's algorithm to enforce this. When is this constraint feasible (i.e., when does a spanning tree containing e_0 exist)?
 - ii. You are told that edge e_0 **must not** be in the MST. Describe how to modify Kruskal's algorithm. Is there always a spanning tree that avoids e_0 ?
 - iii. Suppose e_0 is the unique minimum-weight edge in G but you are forbidden from using it. Describe how to find the MST of G that excludes e_0 . Does simply deleting e_0 and rerunning Kruskal's work?
 - iv. You are given a partial spanning tree F (a forest of $k < n - 1$ edges, all of which must be included). Show how to complete F into a minimum spanning tree by running a modified Kruskal's on the remaining edges.
15. The *multipop stack* is a canonical example showing that expensive operations, when rare, do not ruin the amortized bound.
- A stack supports three operations: **Push**(x) (push one element, cost 1), **Pop** (pop one element, cost 1), and **MultiPop**(k) (pop the top $\min(k, |S|)$ elements, cost = number of elements popped).
- (a) What is the worst-case cost of a single **MultiPop**(k) on a stack of size n ?
 - (b) Show that the naive worst-case analysis gives $O(n^2)$ for n operations starting from an empty stack. (Describe the sequence that achieves this bound under naive analysis.)
 - (c) Use the **accounting method** to prove that the amortized cost per operation is $O(1)$. Specify how many credits you assign to **Push**, **Pop**, and **MultiPop**, and what invariant you maintain.⁸
 - (d) Define a potential function Φ and use the **potential method** to arrive at the same $O(1)$ amortized cost. Show that $\hat{c}_i \leq 2$ for **Push** and $\hat{c}_i \leq 0$ for **Pop**/**MultiPop**.

16. Implementing a queue from two stacks is a classic interview question. Amortized analysis reveals why it is efficient.

Implement a **queue** (FIFO) using two stacks S_{in} and S_{out} , each supporting **Push** and **Pop** in $O(1)$ worst case.

- **Enqueue**(x): push x onto S_{in} .
 - **Dequeue**: if S_{out} is empty, pop all elements from S_{in} and push them onto S_{out} ; then pop from S_{out} .
- (a) Trace this data structure on the sequence: **Enqueue**(1), **Enqueue**(2), **Enqueue**(3), **Dequeue**, **Enqueue**(4), **Dequeue**, **Dequeue**. Show the contents of both stacks at each step.
 - (b) What is the worst-case cost of a single **Dequeue** operation?

⁶Suppose T^* is an MST with bottleneck $b^* = \max_{e \in T^*} w(e)$. Suppose for contradiction there exists a spanning tree T with bottleneck $b < b^*$. Let $e^* \in T^*$ be the bottleneck edge of T^* . Since all edges of T weigh at most $b < w(e^*)$, and T is connected, argue that some edge of T crosses the fundamental cut of e^* with weight $< w(e^*)$, contradicting the optimality of T^* .

⁷Suppose the path in T^* uses an edge e with $w(e) = b$. If some other path P avoids e and has bottleneck $< b$, show that P contains an edge crossing the fundamental cut of e with weight $< b$, contradicting the MST cut property.

⁸Each element can be popped at most once for every time it is pushed.

- (c) Define a potential function $\Phi = |S_{\text{in}}|$ (the number of elements in S_{in}). Use the **potential method** to show that the amortized cost of each **Enqueue** and **Dequeue** is $O(1)$.
- (d) Conclude that n **Enqueue** and **Dequeue** operations starting from an empty structure cost $O(n)$ in total.
17. *The binary counter analysis generalizes beautifully, but adding a reset or decrement operation can break everything.*
- (a) A k -bit binary counter supports **Increment** (cost = bits flipped) and **Reset** (set all bits to 0, cost = k). Using the **aggregate method**, show that m **Increment** operations *without* any **Reset** cost $O(m)$ total.
- (b) Now consider a sequence of m operations, each being either **Increment** or **Reset**. Give a sequence of m operations where the total cost is $\Theta(mk)$. (This shows the $O(m)$ bound breaks once resets are allowed.)
- (c) Suppose instead of resetting to all-zeros, **Reset** resets the counter to its value *before the last increment*. Does the $O(1)$ amortized bound for **Increment** still hold? Prove or disprove.
- (d) Consider a counter that supports both **Increment** and **Decrement** (subtract 1 in binary). Explain why the potential $\Phi = (\text{number of 1-bits})$ no longer yields $O(1)$ amortized cost for both operations together. Give a sequence of increments and decrements where the total cost is $\Theta(mk)$.⁹
18. *Dynamic arrays are ubiquitous in practice. The choice of growth factor critically affects both the amortized cost and memory usage.*

A dynamic array doubles its capacity whenever an element is pushed into a full array. All other pushes cost 1.

- (a) Use the **accounting method** to show that the amortized cost of **Push** is $O(1)$. Assign 3 credits per **Push** and show the invariant: elements in the right half of the array each hold 2 credits, which is exactly enough to pay for the next doubling.¹⁰
- (b) Now suppose the array grows by a **factor of** $r > 1$ (not necessarily 2) when full. For example, $r = 1.5$ means the new capacity is $1.5\times$ the old capacity. Show that the amortized cost of **Push** is still $O(1)$ for any fixed $r > 1$.
- (c) Now consider **shrinking**: when the array becomes less than $1/4$ full, halve its capacity (cost = number of elements copied). Show that the amortized cost of each push and pop is still $O(1)$.¹¹ Why is the threshold $1/4$ (not $1/2$) important for this to work?

⁹Alternate between $0\underbrace{1\cdots 1}_{k-1}$ and $\underbrace{1\cdots 1}_k$.

¹⁰At the moment of doubling from capacity c to $2c$, all c elements in the array need to be copied. Which elements have been holding credits?

¹¹Use $\Phi = 2 \cdot \text{size} - \text{capacity}$ when $\text{size} \geq \text{capacity}/2$, and $\Phi = \text{capacity}/2 - \text{size}$ when $\text{size} < \text{capacity}/2$. Verify $\Phi \geq 0$ in both cases.